

Politecnico di Torino
Department of Electronics and Telecommunications
Master's degree in Mechatronic Engineering



Sensors, Embedded Systems and Algorithms for Service Robotics

2025/2026

Report 3¹

Date, 7/01/2026

Group n° 14

Luigi Muratore — S333098

¹GitHub Repository: <https://github.com/luigimuratore/SES4SR-ros>

Objective

Developing and validating a ROS 2 navigation package for a mobile robot that implements the Dynamic Window Approach (DWA) for real-time obstacle avoidance and dynamic goal chasing, using LiDAR and vision-based AprilTag detection to follow a moving target in both simulation and real-world scenarios.

Goals

In particular the goals were:

- implement a Dynamic Window Approach controller in Python as a ROS 2 node, including trajectory and optimization based on a weighted cost function accounting for heading alignment, velocity, and obstacle clearance;
- design and tune safety mechanisms to prevent collisions by using LiDAR measurements;
- extend the standard DWA formulation by introducing additional cost terms to enable velocity reduction near the goal;
- integrate a goal management system capable of handling dynamic goal updates during execution;
- evaluate the navigation performance in simulation and on a real TurtleBot3 robot by recording ROS topics and computing quantitative metrics, including success rate, tracking time, distances and errors.

Design

Task 1: Dynamic Window Approach DWA

Task 1 implements a ROS 2 node (*Task_1*) for tracking the 3-degree-of-freedom (3-DOF) state of the robot (x, y, θ) in a simulated environment using the Dynamic Window Approach (DWA) for navigation. The node subscribes to odometry data, LiDAR scans, and the goal pose topic where the position of the target are updated.

The main components of the DWA implementation are:

- **Trajectory Sampling:** The robot's possible trajectories are sampled based on its current velocity and acceleration limits.
- **Cost Function:** Each trajectory is evaluated using a weighted cost function that considers heading alignment with the goal, forward velocity, and obstacle avoidance term. In particular the formulation used is:

$$J = \alpha \times \text{heading} + \beta \times \text{vel} + \gamma \times \text{dist}_{\text{obst}}$$

where α , β , and γ are tunable weights for each component, respectively heading, velocity, obstacle avoidance.

Additionally, the nodes handles:

- **Safety Mechanisms:** Safety checks are implemented to prevent collisions by analyzing LiDAR data, filtering them, and stopping the robot when necessary.
- **Goal Management:** The node handles dynamic goal updates, and state conditions between: Goal, Collision, Timeout. Goal state correspond to the tracking phase, so the robot is following the new poses of the movign target; Collision state is triggered when an obstacle is detected within a predefined safety distance; Timeout state is activated when the robot fails to enable Goal phase within a specified time limit.
- **Distance information topic:** The node publishes detailed information about the robot's pose, in particular the distance to the goal, in a dedicated topic: `dwa_feedback`.
- **Metrics topic:** The node publishes the performance metrics such as success rate, tracking time, and average distance to the goal, minimum distance to the goal, in a dedicated topic: `metrics`.
- **Controller:** The controller computes and publishes velocity commands to the robot based on the selected optimal trajectory.

ROS2 Node: Task_1

The node initializes topics and parameters as follows:

- **Subscribed Topics:**
 - `/scan`: to take LiDAR scan data for obstacle detection;
 - `/odom`: to receives odometry data for the robot's current position and velocity;
 - `/dynamic_goal_pose`: to have dynamic goal position updates during execution in simulation;
 - `/camera/landmarks`: to have dynamic goal position updates during execution in real-world with the camera.
- **Published Topics:**
 - `/cmd_vel`: to send velocity commands to the robot in order to control its movement;
 - `/dwa_feedback`: to publish feedback on the robot's distance to the goal;
 - `/metrics`: to publish performance metrics of the navigation task.
- **Parameters:**
 - DWA parameters:
 - * *alpha*: for the heading alignment;
 - * *beta*: for the velocity;
 - * *gamma*: for the obstacle avoidance;
 - Controller and metrics parameters:
 - * *control_rate* (15.0): to set the control loop frequency;
 - * *collision_radius* (0.20): to set the Robot collision radius;
 - * *num_ranges* (18): to set the number of LiDAR ranges to sample;
 - * *max_lidar_range* (3.5): to set the maximum LiDAR detection range;
 - * *max_linear_vel* and *max_angular_vel* (0.21, 3.0): to set the maximum linear and angular velocity;
 - * *desired_distance* (0.5): to set the desired distance to maintain from the goal;
 - * *tracking_threshold* and *max_bearing_error* (2.0, 90): to set the distance threshold and maximum acceptable bearing error to stay in tracking state;

In particular, the DWA parameters *alpha*, *beta*, and *gamma* were tuned through several tests to achieve a balance between heading alignment and velocity behavior considering obstacle avoidance.

The main objective of the node is make the robot continuously follows a moving target by receiving its position through the `/dynamic_goal_pose` (in simulation) and `/camera/landmarks` (in real-world) topic and computing the target's heading from its velocity direction (change in position over time). At each control cycle running at 15 Hz, the controller uses DWA to generate velocity commands that balance three different weights: moving toward the target position, maintaining desired velocity shapes, and avoiding obstacles detected by LiDAR data. The system tracks whether the robot is successfully following the target by checking if it maintains a close distance (within 2 meters), keeps its heading aligned with the target's direction (within 90 degrees bearing error), and is not moving away from the target. When these tracking conditions are met, the controller accumulates tracking time and records distance and bearing errors for performance evaluation. The controller continuously publishes velocity commands to `/cmd_vel` and monitors for safety conditions like obstacle collisions, automatically stopping the robot if obstacles get too close. Every 50 steps, it publishes metrics including the percentage of time successfully tracking, root-mean-square errors RMSE for distance and bearing, and statistics about obstacle proximity to evaluate the controller's performance.

Task 2: Variants of the DWA

Task-2 extends the Task-1 architecture to a more defined velocity shape, by implementing additional cost terms in the DWA cost function to encourage velocity reduction when approaching the goal. This is achieved by adding a new term to the cost function that penalizes high velocities as the robot nears its target, promoting smoother and safer stops. The modified cost function is defined as:

$$J = \alpha \times \text{heading} + \beta \times \text{vel}' + \gamma \times \text{dist}_{\text{obst}} + \delta \times \text{dist}_{\text{target}}$$

It has been also tried to introduce a trapezoidal velocity profile to achieve smoother motion characteristics. Additionally than adding a penalty term for high velocities near the goal, the modification of the velocity scoring mechanism has been divided into three distinct phases based on distance metrics:

1. **Acceleration Phase:** linearly ramps velocity from zero to maximum;
2. **Cruise Phase:** maintains maximum velocity when sufficient distance remains;
3. **Deceleration Phase:** applies a linear velocity reduction proportional to remaining distance to the goal.

This trial has shown bad performance, cause it made the algorithm more complex without significant improvements in navigation behavior. The robot struggled to adapt its speed effectively, especially in dynamic scenarios where the target's movement required frequent speed adjustments.

The most efficient way remained the tuning of the penalty term for high velocities near the goal.

Task 3: Real Robot Deployment

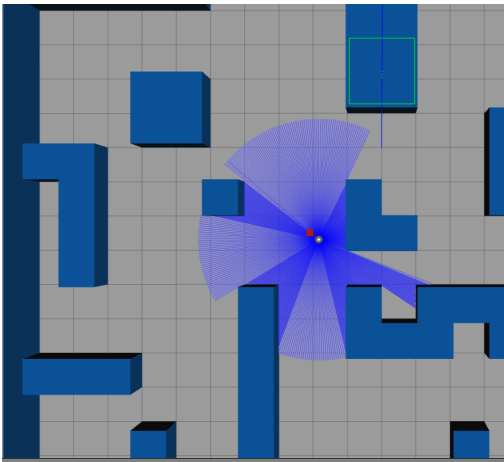
Task 3 focuses on deploying the DWA navigation system on a real TurtleBot3 robot.

The architecture remains the same of Task 1 and Task 2, with necessary adjustments for the target pose update topic. In this case, the robot receives the moving target's position from its external camera system that detects and tracks visually the landmarks.

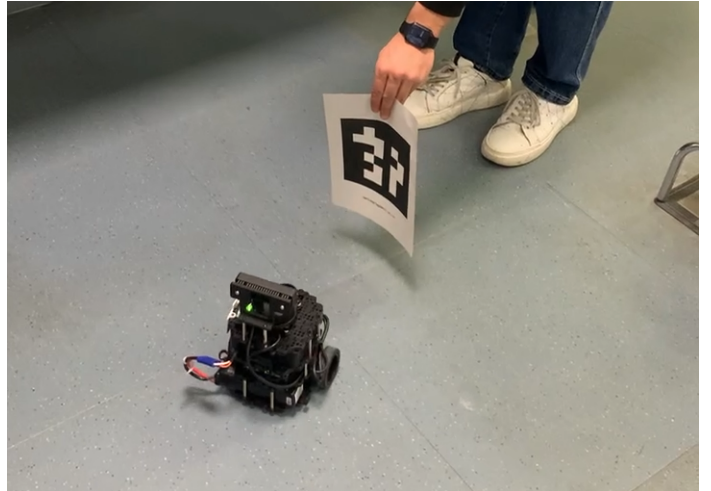
Experiment

The experiments were conducted in both simulated and real-world environments to evaluate the performance of the implemented Dynamic Window Approach (DWA) navigation system.

The simulation was performed using Gazebo with a TurtleBot3 robot model (Fig. 1a), while the real-world tests were carried out on an actual TurtleBot3 robot in a controlled indoor environment (Fig. 1b).



(a) Simulated map in Gazebo with TurtleBot3 robot navigating towards a moving goal (red cube).



(b) Real-world test setup with TurtleBot3 following a moving goal in the laboratory.

Figure 1: Experimental setup: simulation and real-world experiments.

Simulation tests in Gazebo

After implementing the DWA navigation node, several simulation tests were conducted to evaluate its performance in tracking, but the most trial has been performed to tune the DWA parameters (α , β , γ) for optimal navigation behaviour in terms of heading the target, speed and obstacle avoidance.

The Fig. 2 shows the simulations tests results in terms of odometry errors in heading, velocity and obstacle during the navigation task, so the bad optimized choice of parameters (α , β , γ): Fig. 5a shows the odometry error both in heading and velocity due to wrong values of α and β , Fig. 5b shows the odometry error mainly in heading due to wrong value of α , while Fig. 2c shows the odometry error mainly in velocity due to wrong value of β .

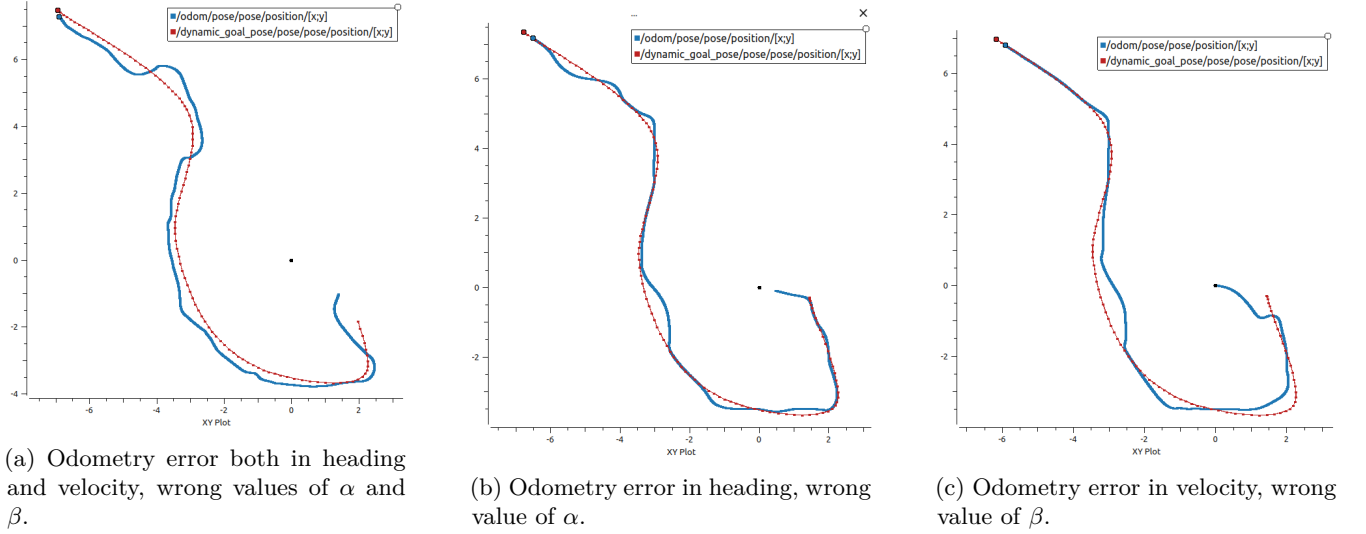


Figure 2: Odometry results: position (x,y) of the moving target over time in red curve, position (x,y) of the robot over time in blue curve.

Several tests has been done to also find a good velocity profile: Fig. 3 hows three different velocity profiles obtained during the simulation tests. Fig. 3a shows a velocity profile where the robot moves too slowly, Fig. 3b shows a velocity profile where the robot always moves at the maximum velocity cause it never gets close to the target, while Fig. 3c shows a good velocity profile where the robot is able to adjust its speed based on the distance to the moving target.

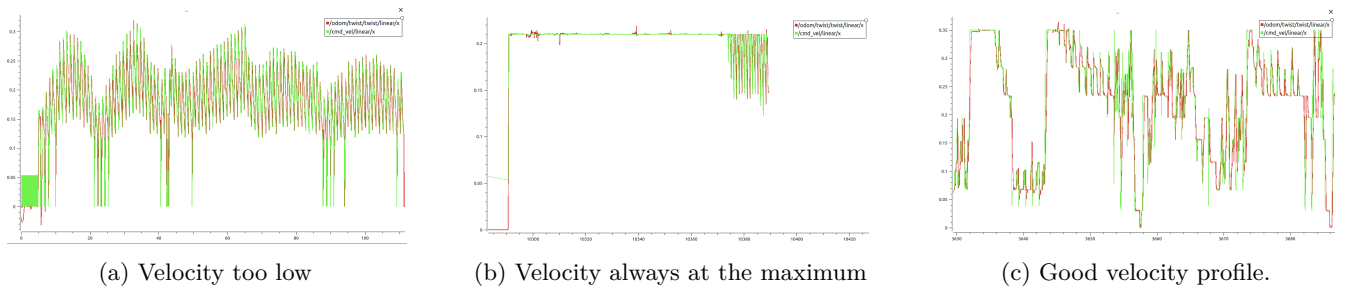


Figure 3: Velocity profiles during simulation tests.

The best configuration found is:

- $\alpha = 0.12$
- $\beta = 1.0$
- $\gamma = 0.4$

These values provided a good balance between heading alignment, velocity, and obstacle avoidance, allowing the robot to effectively track the moving target while navigating around obstacles. Fig. 4a shows the odometry results during the simulation tests with the best DWA parameters configuration while Fig. 4b shows the best velocity profile found after several trials.

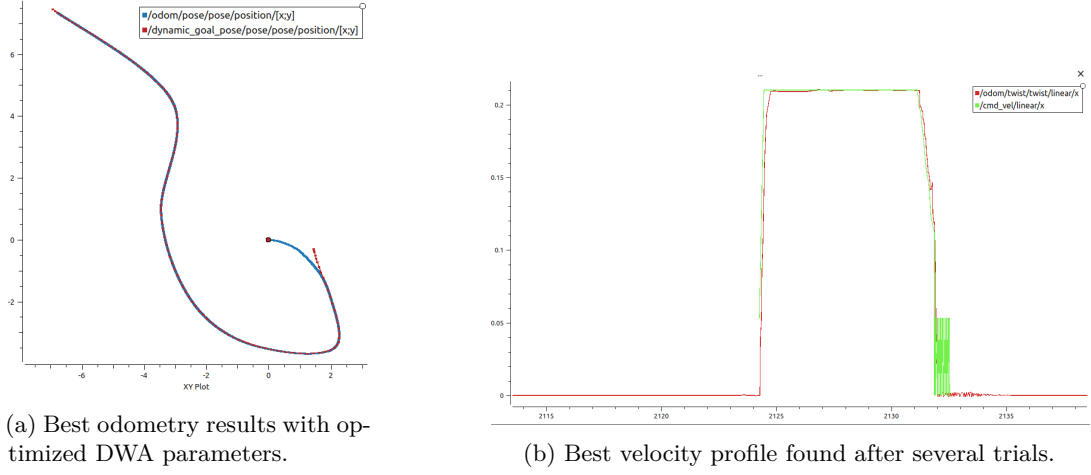
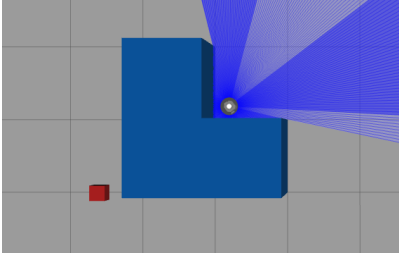


Figure 4: Best results in terms of odometry and velocity profile during simulation tests.

During the simulation tests, collision detection was also evaluated. The system successfully detected collisions with obstacles in the environment, as shown in Fig. 5, where the robot encounters an obstacle, in particular a wall due to a wrong choice of the parameter γ . The collision event is logged in the terminal while the robot is stopped.



(a) Simulated obstacle crash in the map.

```
[INFO] [1767107254.813212918] [task_1]: Min laser range: 0.18
[INFO] [1767107254.864115376] [task_1]: Event: Collision
```

(b) Collision event call in the terminal.

Figure 5: Collision detection during simulation tests.

When the robot is in *Goal* state, the controller is able to handle two different condition: *Tracking*, *Lost*.

In *Tracking* condition, the robot is able to follow the moving target closely, maintaining a small distance and keeping its heading aligned with the target's direction. The controller continuously adjusts the robot's velocity commands to ensure it stays on course towards the target while avoiding obstacles.

In *Lost* condition, the robot is unable to maintain effective tracking of the moving target. This can occur if the target moves too quickly, if there are significant obstacles in the environment, or if the robot's sensors fail to detect the target accurately. When in this state, the controller may attempt to re-acquire the target by adjusting its heading and searching for the target's last known position. If the robot remains in the *Lost* condition for an extended period, it should stop moving to avoid collisions or further deviation from the target's path. Fig. 6 shows the terminal output when the robot is in *Lost* condition during simulation tests.

```
[INFO] [1767105774.039265080] [task_1]: Min laser range: 0.80
[INFO] [1767105774.094886438] [task_1]: [LOST] v=0.21, w=0.80 | dist_target=2.35m, bearing=-74.5°
[INFO] [1767105774.104630253] [task_1]: [LOST] v=0.21, w=0.91 | dist_target=2.33m, bearing=-76.8°
[INFO] [1767105774.231990943] [task_1]: [LOST] v=0.21, w=1.03 | dist_target=2.32m, bearing=-79.7°
```

Figure 6: Robot in *Lost* condition during simulation tests.

Real tests in the Laboratory

We conducted several real-world tests in our laboratory using the TurtleBot3 robot.

Due to some problems with the real-world test node, we are not able to provide the comparison between the odometry of the robot and the position of the moving target over time; all the trials demonstrations video are still be visible at the GitHub repository² where all the video dimonstrations are posted.

Fig Fig. 7 shows the odometry results obtained during the real-world tests, while Fig. 8 shows the linear and angular velocity profiles of the robot during the navigation task.

The velocity profiles show that the robot is able to adjust its speed based on the distance to the moving target, similar to the simulation tests.

The tracking performance in the real-world tests was generally good, not as the simulation tests, but still with the robot successfully following the moving target while avoiding obstacles.

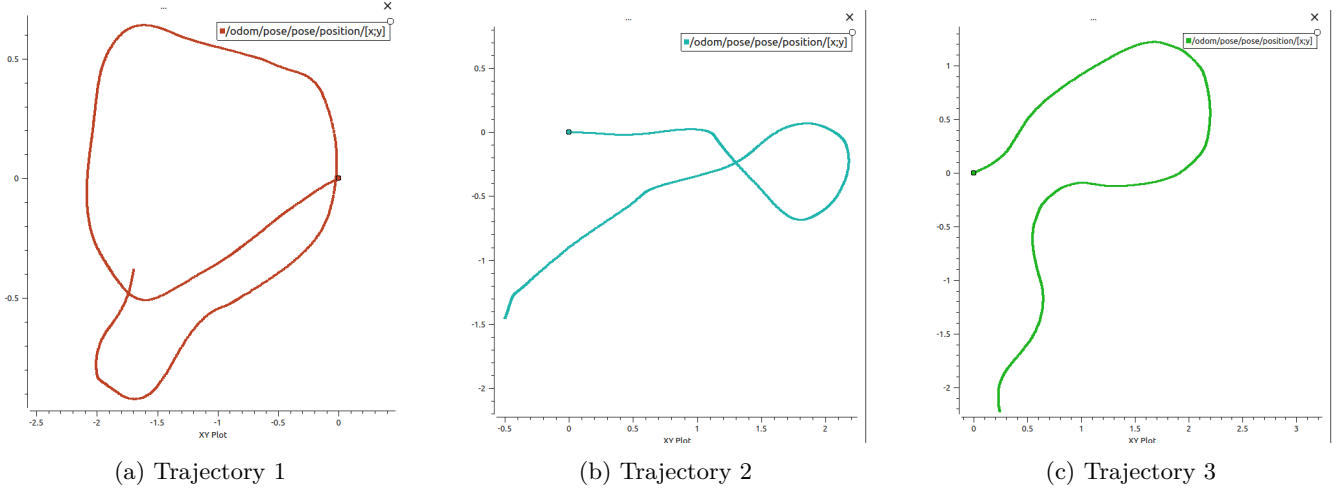


Figure 7: Odometry from three trajectories obtained during real-world tests.

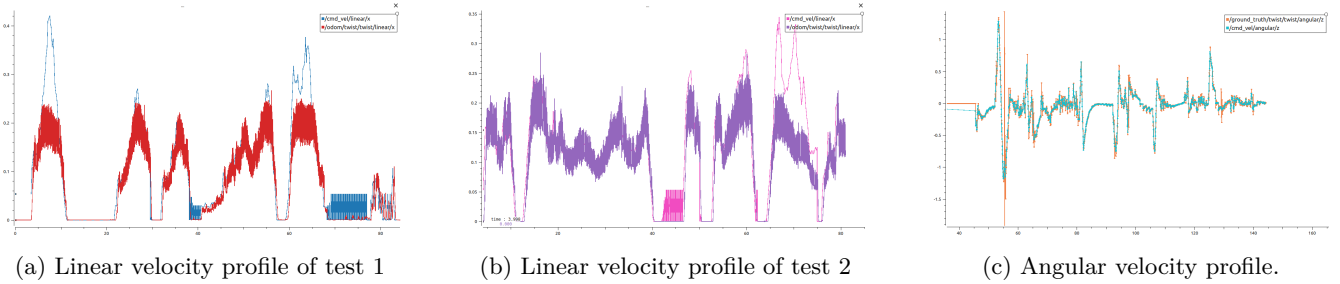


Figure 8: Velocity profiles from the three tests obtained during real-world tests.

²https://github.com/luigimuratore/SES4SR-ros/blob/main/src/lab05_pkg/lab05_pkg/README.md#demonstrations

Results and Discussion

The implemented DWA navigation system was evaluated through a series of simulation and real-world tests. The performance metrics were collected and analyzed to assess the effectiveness of the navigation strategy in tracking a moving target while avoiding obstacles.

Metrics

The performance of the implemented DWA navigation system was evaluated using the following metrics:

- **Success Rate:** the percentage of trials where the robot successfully reached the moving target without collisions within a specified time limit;
- **Tracking Time:** how long the moving target has been chasing correctly, so the amount of time (percentage over the entire duration of the experiment) during which the robot did not lose the target robot.
- **RMSE - Distance to Goal:** The root mean square error between the robot's position and the moving target's position;;
- **RMSE - Bearing to Goal:** The root mean square error of the bearing angle between the robot's heading and the moving target;
- **Average Distance to Goal:** The mean distance between the robot and the moving target during the navigation task;
- **Minimum Distance to Goal:** The closest distance achieved between the robot and the moving target during the navigation task.

Typical metrics obtained during simulation are reported in the table 1: the first column shows the results obtained with poorly tuned DWA parameters, while the second column shows the results obtained with well-tuned DWA parameters after several trials and tests.

Metric	Bad DWA params	Good DWA params
Success Rate	90%	100%
Tracking Time	85%	100%
RMSE - Distance to Goal	0.58 m	0.24 m
RMSE - Bearing to Goal	25.9°	12.10 °
Average Distance to Goal	2.25 m	2.21 m
Minimum Distance to Goal	0.45 m	0.19 m

Table 1: Performance metrics from simulation tests.

Metrics obtained during the three simulations in real-world with the optimized DWA parameters are reported in the table 2: in trajectory 2 the robot crashed against an obstacle and remained stuck for a while, causing a lower success rate, while in the other two trajectories the robot was able to follow the moving target successfully without collisions but with some interactions in tracking due to the unstable paper we used as landmark.

Metric	Trajectory 1	Trajectory 2	Trajectory 3
Success Rate	100%	80%	100%
Tracking Time	70%	52%	65%
RMSE - Distance to Goal	0.4 m	0.6 m	0.6 m
RMSE - Bearing to Goal	16.7°	42.1°	21.3°
Average Distance to Goal	0.6 m	2.3 m	1.1 m
Minimum Distance to Goal	0.1 m	0.23 m	0.17 m

Table 2: Performance metrics from three real-world test trajectories.

Fig. 9 shows sample metrics output in the terminal during tests. Fig. 9a shows the metrics obtained with well-tuned DWA parameters, while Fig. 9b shows the metrics obtained with poorly tuned DWA parameters, resulting in lower success rates and longer tracking times.

```
data: 'Track:100.0% T:140.0s DistRMSE:0.244m BearRMSE:12.1° ObstAvg:2.20m Min:0.19m'
```

(a) Metrics on terminal with good DWA parameters.

```
data: 'Track:86.7% T:100.0s DistRMSE:0.587m BearRMSE:25.9° ObstAvg:2.22m Min:0.45m'
```

(b) Metrics on terminal with bad DWA parameters.

Figure 9: Metrics published during simulation tests.

In any case the real-world tests showed slightly lower performance compared to the simulation tests, in term of success rate and tracking time. In particular, the average distance to goal was around 1 meter, and the minimum distance to goal was around 0.1 meters, indicating that the robot was able to follow the moving target closely, but the tracking time was really lower than the simulation on Gazebo. We reached around 60% of tracking time cause sometimes the robot lost the target and stayed still for some seconds before re-acquiring it.

Conclusion

The implemented DWA navigation system demonstrated effective performance in autonomously tracking a moving target while avoiding obstacles.

The system successfully integrated dynamic goal updates, obstacle detection, and velocity command generation to achieve robust navigation behavior.

The evaluation metrics indicated a high success rate, efficient tracking times, and close proximity to the moving target across various test scenarios, especially in simulation.

However, real-world tests revealed some challenges, including reduced tracking time and occasional loss of the target, likely due to environmental variability.

Overall, the project provided valuable results into the application of DWA for dynamic target tracking in mobile robot.